

Web App QA & Debug Report

Comprehensive Technical Audit & Stakeholder Briefing on AI-Assisted ('Vibe-Coded') SaaS Platform

Candidate Name:	Monali Agarwal	Date of Audit:	September 29, 2026
Target Role:	Automation & QA Developer	Target Web App:	PulseFeed SaaS (AI Vibe-Coded Prototype)
Tech Stack Assessed:	Django 5.x, REST APIs, SQLite, Bootstrap 5, JS	Testing Framework:	Django TestRunner, Python Requests, Browser Subagent

1. Executive Summary for Non-Technical Stakeholders

Context: The evaluated web application, *PulseFeed*, is a content publishing platform recently developed using AI-assisted ('vibe coding') prompt tools. While the user interface appears polished with modern cards, responsive feeds, and rich text areas, end-to-end quality assurance uncovered **seven critical to medium-severity defects** that make the application unsafe and unreliable for public commercial deployment.

The Vibe-Coding Paradox: AI-assisted code generators excel at producing visual structure (HTML/CSS) and basic database models. However, they routinely suffer from *context blindness*—omitting essential ownership checks, neglecting data persistence during partial updates, and relying entirely on insecure front-end validation. In this application, any registered user can permanently delete another user's content, unescaped malicious scripts can execute directly inside users' browsers, and editing an article silently destroys all its categorizing tags.

2. Bug Report Table (Section 5 Required Template)

The following table documents all identified issues adhering strictly to the required assessment schema, formatted for both technical engineering teams and project stakeholders:

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	Insecure Direct Object Reference (IDOR) on Article Deletion API Endpoint: /api/articles/<id>/delete/	1. Log in as user 'monali'. 2. Open article #1 authored by 'john_doe'. 3. Send POST/DELETE to /api/articles/1/delete/ (via UI button or API).	Expected: Server returns HTTP 403 Forbidden. Actual: Server returns HTTP 200 OK and permanently deletes John's article.	Critical	Backend checks authentication but lacks authorization logic: omitted article.author == request.user.
2	Stored Cross-Site Scripting (XSS) via Unsanitized Story Body View: article_detail.html	1. Log in and open 'New Story'. 2. In Content body, enter <script>alert('XSS')</script>. 3. Publish and view article.	Expected: Script tags are escaped or sanitized. Actual: JavaScript executes immediately in the reader's browser.	Critical	Django template uses the safe filter on un-sanitized user input without Bleach or DOMPurify.
3	Silent Tag Purging & Data Loss During Article Updates Controller: article_edit_view	1. Create article with tags 'ai, security'. 2. Click 'Edit Article'. 3. Modify only title and click 'Save Changes'.	Expected: Existing tags are preserved. Actual: All tags are stripped; tags count drops from 2 to 0.	High	Template fails to pre-fill existing tags in edit input; controller executes unconditional article.tags.clear().

4	Unhandled IntegrityError (HTTP 500) on Duplicate Registration View: register_view	1. Navigate to /register/. 2. Submit username that already exists (e.g., 'monali'). 3. Submit form.	Expected: HTTP 400 with user-friendly alert 'Username already taken'. Actual: HTTP 500 crash with raw unhandled IntegrityError traceback.	High	Direct call to <code>User.objects.create_user()</code> without uniqueness pre-check or try/except <code>IntegrityError</code> .
5	Session State Desync & Missing Cache-Control on Logout View: logout_view	1. Sign in as user. 2. Click 'Log Out' from dropdown. 3. In browser, click the 'Back' navigation button.	Expected: Cached private screens are purged. Actual: Browser renders authenticated view and allows action retries.	Medium	Views omit <code>Cache-Control: no-store, no-cache</code> headers; JS logout removes client token but leaves session alive.
6	Duplicate Article Creation on Rapid Double-Click Form: editor.html	1. Fill New Story form. 2. Rapidly double-click 'Publish Story' button. 3. Inspect database or home feed.	Expected: Single article created; button disabled on first click. Actual: Two identical articles created with duplicate titles.	Medium	Lack of client-side button debouncing/disable state, coupled with missing backend request idempotency tokens.
7	Severe Dark Mode Contrast Failure (WCAG 2.1 AA Violation) CSS: static/css/style.css	1. Open feed on desktop or mobile. 2. Click moon icon in navbar to toggle Dark Theme. 3. Observe story card summaries.	Expected: Text color dynamically switches to high contrast (#e2e8f0, >4.5:1 ratio). Actual: Body text stays dark grey (#333333), yielding 1.4:1 contrast ratio.	Low	AI-generated CSS inverted <code>background-color</code> to #1e1e1e but omitted overriding child <code>.card-text</code> colors.

3. Deep-Dive Root-Cause Analysis (Issue #1: IDOR Vulnerability)

Requirement: Pick ONE issue and write a 5–10 sentence non-technical root-cause analysis explaining what is happening, why, and how to fix it.

What is happening: Currently, any logged-in user can permanently delete any other user's articles simply by knowing or guessing the article's identification number. During testing, logging in as user 'monali' allowed complete deletion of user 'john_doe's article (#1) with zero permission warnings.

Why it is happening: When the non-developer prompted the AI coding tool to build an article deletion feature, the AI generated a basic security check verifying only that the visitor was logged in (`request.user.is_authenticated`). However, the AI forgot to verify whether the logged-in user is actually the *original author* of the article being deleted. In software architecture, this flaw is known as an Insecure Direct Object Reference (IDOR), and it is one of the most common pitfalls of AI-generated code because LLMs routinely satisfy the 'happy path' without generating defensive access controls.

How to fix it: The fix requires adding a strict ownership validation guard in the backend deletion view before any database modification occurs. Specifically, the code must check: `if article.author != request.user: return JsonResponse({'error': 'Forbidden'}, status=403)`. Additionally, the front-end user interface must conditionally hide the 'Delete' button unless the current user matches the author, and automated integration tests should be added to our deployment pipeline to guarantee that cross-user deletions are permanently blocked.

4. Secondary Security Analysis (Issue #2: Stored Cross-Site Scripting)

What is happening: When authors publish stories containing malicious HTML or JavaScript code, the platform saves the script verbatim and executes it inside the web browser of every user who subsequently views the story.

Why it is happening: The developer instructed the AI to 'support rich formatting and HTML.' The AI responded by applying Django's built-in `|safe` filter in the HTML template (`{{ article.body|safe }}`). The `safe` filter explicitly commands the web server to disable automatic security escaping, trusting user input entirely. Because there is no intermediary sanitization library, an attacker can steal user cookies or session tokens.

How to fix it: Remove the raw `|safe` template tag and install an established HTML sanitization library such as `bleach` or `nh3`. All incoming story content must be stripped of dangerous tags (`<script>`, `onerror`, `onload`) before database persistence, allowing only benign tags (`<p>`, `<b>`, `<code>`).

5. Automated QA Verification & Regression Harness

To provide indisputable proof of these findings and guarantee they never regress once patched, an automated regression test suite was constructed using Django's TestCase harness in tests/test_qa_bugs.py.

Test Execution Results:

- test_bug_01_idor_unauthorized_deletion: PASS (Proves cross-user deletion occurs and returns HTTP 200).
- test_bug_02_stored_xss_rendering: PASS (Proves raw script tags pass unescaped to DOM).
- test_bug_03_silent_tag_purging_on_edit: PASS (Proves tag count drops from 2 to 0 on standard edit).
- test_bug_04_unhandled_integrity_error_on_duplicate_registration: PASS (Proves unhandled IntegrityError crash).
- test_bug_06_duplicate_creation_on_double_submit: PASS (Proves duplicate rows created on concurrent submit).

Overall Test Suite Status: 5 tests executed in 9.248s — 100% defect reproducibility confirmed.

6. Strategic Recommendations for Product Leadership

- 1. Enforce Server-Side Business Validation:** AI prompt tools often generate validation only in HTML5 (e.g. required attributes). All authorization, data uniqueness, and sanitization must be enforced on the server.
- 2. Implement Object-Level Permissions:** Transition from basic boolean checks (is_authenticated) to granular ownership verification rules on all write/update/delete operations.
- 3. Integrate Automated Pre-Commit QA Suites:** Every AI-generated code change should be gated by an automated test run that specifically exercises boundary conditions, negative authentication flows, and load testing.
- 4. Establish Accessibility (a11y) Linting:** Incorporate automated color contrast checks to ensure dynamic themes comply with international accessibility standards (WCAG 2.1 AA).